



Vijay Academy Presents



Savitribai Phule Pune University (SPPU)

Web Technology (WT)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 3. Java Servlets and XML

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions



Subscribe The Vijay Academy For Free Engineering Content

Unit 3. Java Servlets and XML

Syllabus Topics :

Section A — Java Servlets

1. Servlet Architecture Overview
2. A "Hello World" Servlet
3. Servlets Generating Dynamic Content
4. Servlet Life Cycle
5. Parameter Data (doGet & doPost)
6. Sessions
7. Cookies
8. URL Rewriting
9. Other Servlet Capabilities
10. Data Storage
11. Servlets Concurrency
12. Databases (MySQL) and Java Servlets

Section B — XML

13. XML Documents and Vocabularies
14. XML Declaration & Structure
15. XML Namespaces
16. DOM Based XML Processing
17. Transforming XML Documents
18. DTD — Schema, Elements & Attributes

Section C — AJAX

19. Introduction to AJAX
20. Working of AJAX



1. Servlet Architecture Overview

A **Servlet** is a **Java program** that runs on a **web server** and handles requests from a client (browser) and sends back a response.

Servlet Architecture means — how the client, web server, servlet container, and servlet all work together.

Example (Real Life)

Think of it like a **restaurant**:

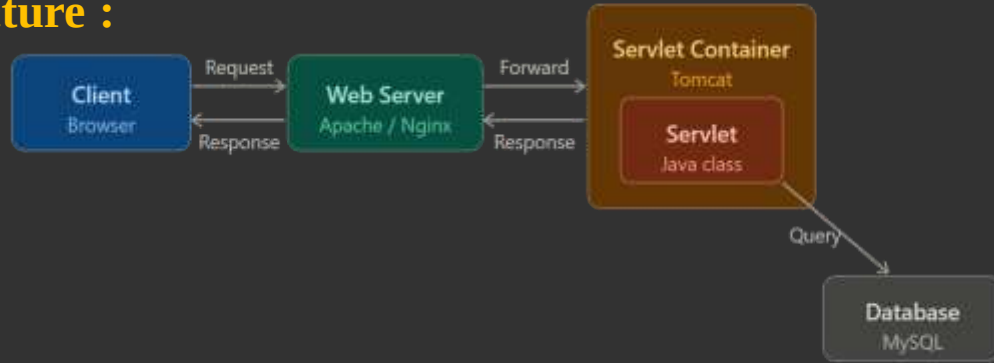
Client (Browser) = Customer who orders food

Web Server = Waiter who takes the order

Servlet Container (Tomcat) = Kitchen

Servlet = Chef who prepares the food and sends it back.

Servlet Architecture :



1. Client sends HTTP request.

You type a URL in browser and hit enter. Browser sends a request like "Hey, give me this page!"

2. Web Server receives it.

Apache/Nginx gets the request. It forwards it to Tomcat.

3. Servlet Container finds the right Servlet.

Tomcat looks at the URL and matches it to the correct Servlet class using a config file called **web.xml**.

4. Servlet processes & talks to DB.

The Servlet runs its Java code — reads form data, queries MySQL if needed and prepares the answer.

5. Response goes back.

Servlet sends HTML/data back → **Servlet Container** → Web Server → Browser shows the page.



2. A "Hello World" Servlet

A **Hello World Servlet** is the simplest servlet program that just prints "**Hello World**" on the browser when you open it.

It shows you the basic structure of how to write a servlet.

Basic Structure of a Servlet:

Every servlet:

extends HttpServlet class.

HttpServlet is a **ready made class** by Java. It already has lots of built-in HTTP handling code.

overrides doGet() or doPost() method.

doGet() → runs when browser **opens a URL** normally.

doPost() → runs when browser **submits a form**.

You **override** means you write your own version of this method.

Uses PrintWriter to send output to browser.

HttpServlet Class :

HttpServlet is a **ready-made Java class** provided by Java. When you write a servlet, you **extend** this class. It already has all the basic HTTP methods built in — you just override what you need.

Important Methods of HttpServlet :

Method	When it runs
doGet()	When browser sends GET request
doPost()	When browser sends POST request
doPut()	When client sends PUT request
doDelete()	When client sends DELETE request
service()	Automatically calls doGet/doPost



Example :

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class HelloWorldServlet extends HttpServlet {
```

```
    public void doGet(HttpServletRequest request,  
                      HttpServletResponse response)
```

```
    throws ServletException, IOException {
```

```
        // Step 1: Set response type
```

```
        response.setContentType("text/html");
```

```
        // Step 2: Get writer to send output
```

```
        PrintWriter out = response.getWriter();
```

```
        // Step 3: Print Hello World
```

```
        out.println("<h1>Hello World!</h1>");
```

```
    }
```

```
}
```



java.io. → gives us **PrintWriter** to write output

javax.servlet. → gives basic servlet classes

javax.servlet.http. → gives **HttpServlet, Request, Response.**

```
public class HelloWorldServlet extends HttpServlet
```

We create our class HelloWorldServlet

extends HttpServlet → we are saying "**I am a Servlet**"

doGet() Method:

```
public void doGet(HttpServletRequest request,
```

```
    HttpServletResponse response)
```

throws ServletException, IOException

doGet() runs when **browser opens this URL**

request → carries data **coming from browser**

response → used to **send data back to browser**

throws → just means errors may occur, Java needs to know

Step 1 — setContentType

```
response.setContentType("text/html");
```

Tells browser "**I am sending HTML**"

Step 2 — getWriter

```
PrintWriter out = response.getWriter();
```

Gets a **pen** to write output

out is now ready to write anything to browser

Step 3 — println

```
out.println("<h1>Hello World!</h1>");
```

Actually **writes Hello World** on the browser

Browser receives this HTML and displays it

3. Servlets Generating Dynamic Content

Definition :

Static content = same page for everyone (like a plain HTML file)

Dynamic content = page **changes** based on user, time, input etc.

What makes it Dynamic here?

Every time you refresh the page:

`new java.util.Date()` gives **current time**

So the output **changes every time**

That is dynamic content!

Static vs Dynamic

Static

Plain HTML file

Same for everyone

No Java needed

Cannot read DB

Dynamic

Servlet / JSP

Different for each user

Java code runs on server

Can read from database

4. Servlet Life Cycle

Servlet Life Cycle means — what happens to a servlet from the time it is **born** to the time it is **destroyed**.

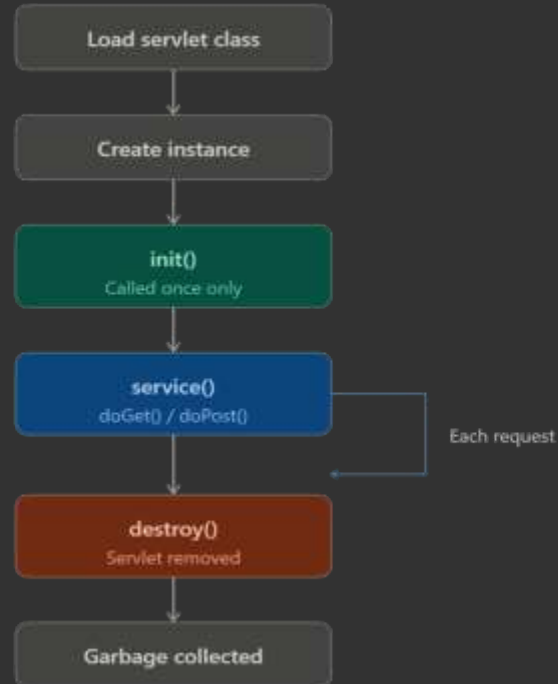
3 main stages:

init() → Servlet is born

service() → Servlet does work

destroy() → Servlet is killed

Diagram :





Explanation :

1. **Load Servlet Class** - Servlet Container reads your .class file into memory.

2. **Create Instance** - Servlet Container creates ONE object of your servlet class.

3. **init()**

Runs **only once** when servlet starts.

Used for setup work — like opening DB connection.

Ex- Like a chef arriving and setting up the kitchen.

4. **service()**

Runs **every time** a request comes.

It checks — GET or POST? Then calls doGet() or doPost()

Ex- Like chef cooking for every new customer.

5. **destroy()**

Runs **only once** when servlet is shutting down.

Used for cleanup — closing DB connections etc.

Ex- Like chef cleaning up and going home.

6. Garbage Collection -

Servlet object is no longer needed. Java's Garbage Collector removes it from memory.

5. Parameter Data (doGet & doPost) :

What is Parameter Data?

When a user fills a form on a webpage and clicks submit — that data (name, password, age etc.) is called **Parameter Data**.

Key Points :

Data comes FROM the browser TO the servlet

It comes through HTML forms

Every form field has a **name** — servlet uses that name to get value

Method used → `request.getParameter("name")`

Returns a **String** always

If field is empty → returns null



HTML Form Example

```
<form action="MyServlet" method="get">  
  Name: <input type="text" name="username">  
  Age: <input type="text" name="age">  
  <input type="submit" value="Submit">  
</form>
```

Servlet gets this data using:

```
String name = request.getParameter("username");  
String age = request.getParameter("age");
```



doGet()

Definition: doGet() handles **GET requests** — when data is sent through the **URL**.

Points:

Called when user types URL or clicks a link.

Data is visible in URL like → servlet?name=Ram&age=20

Data is **not secure** — anyone can see it

Has **size limit** — can't send large data

Used for **reading/fetching** data

Faster than POST

Example :



```
public void doGet(HttpServletRequest request,  
    HttpServletResponse response)  
throws ServletException, IOException {
```

```
// Get data from URL
```

```
String name = request.getParameter("username");
```

```
String age = request.getParameter("age");
```

```
response.setContentType("text/html");
```

```
PrintWriter out = response.getWriter();
```

```
out.println("<h2>Hello " + name + "!</h2>");
```

```
out.println("<p>Your age is: " + age + "</p>");
```

```
}
```

URL looks like:

<http://localhost:8080/MyServlet?username=Ram&age=20>

Part

Meaning

http://

Protocol — how data travels

localhost

Your own computer (server)

8080

Port number — Tomcat runs here

/MyServlet

Name of your Servlet

?

Marks start of data/parameters

username=Ram

Field name = Value

&

Separator between two parameters

age=20

Second field name = Value

doPost()

Definition: doPost() handles **POST requests** — when data is sent **inside the request body**, not in URL.

Points:

Called when user submits a form with method="post"

Data is **NOT visible** in URL — more secure

No size limit — can send large data

Used for **sending/saving** data (login, registration)

Slightly slower than GET

Used for passwords, personal data.

Example :



```
public void doPost(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
```

```
    // Get data from form
    String name    = request.getParameter("username");
    String password = request.getParameter("password");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    if(name.equals("admin") && password.equals("1234")) {
        out.println("<h2>Login Successful!</h2>");
    } else {
        out.println("<h2>Invalid Credentials!</h2>");
    }
}
```

HTML Form:

```
<form action="MyServlet" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>
```

Difference — doGet() vs doPost()

Point	doGet()	doPost()
Data in URL	Yes — visible	No — hidden
Security	Less secure	More secure
Data size	Limited (~2KB)	No limit
Used for	Fetching data	Sending/saving data
Bookmarkable	Yes	No
Form method	method="get"	method="post"
Example use	Search, view page	Login, registration
Speed	Faster	Slightly slower



6. Sessions :

What is a Session?

HTTP is a **stateless protocol** — meaning the server **forgets** who you are after every request.

Session solves this problem — it **remembers** the user across multiple requests.

Real Life Example

Think of a **shopping mall**:

You enter mall → security gives you a **token/card**

You go to different shops — each shop scans your card

Mall knows it's YOU everywhere

You leave mall → card is destroyed

That card = **Session**

Key Points

Session stores user data on the **server side**

Each user gets a **unique Session ID**

Session ID is sent to browser automatically

Server uses Session ID to identify the user

Session has a **timeout** — expires after some time (default 30 min)

Used for — login, shopping cart, user preferences

6. Sessions :

Important Session Methods

Method	Meaning
<code>request.getSession()</code>	Create or get existing session
<code>session.setAttribute("key", value)</code>	Store data in session
<code>session.getAttribute("key")</code>	Get data from session
<code>session.invalidate()</code>	Destroy session (logout)
<code>session.getId()</code>	Get unique session ID
<code>session.setMaxInactiveInterval(sec)</code>	Set timeout time

7. Cookies

What is a Cookie?

A **Cookie** is a small piece of data that the **server sends to the browser** and the browser **saves it** on the client side (your computer).

Next time you visit — browser sends that cookie back to server automatically.

Real Life Example

Think of a **loyalty card at a coffee shop**:

First visit → shop gives you a **stamp card**

You keep that card in your wallet (browser)

Next visit → you show your card

Shop knows you are a returning customer

That stamp card = **Cookie**

Key Points

Server **creates** cookie and sends to browser

Browser **stores** it and sends back with every request

Cookie has a **name, value and expiry time**

Not good for sensitive data (passwords etc.)

Max size → **4KB**

7. Cookies

Important Cookie Methods

Method	Meaning
<code>new Cookie("name", "value")</code>	Create a cookie
<code>response.addCookie(cookie)</code>	Send cookie to browser
<code>request.getCookies()</code>	Get all cookies from browser
<code>cookie.getName()</code>	Get cookie name
<code>cookie.getValue()</code>	Get cookie value
<code>cookie.setMaxAge(seconds)</code>	Set expiry time

8. URL Rewriting

What is URL Rewriting?

URL Rewriting is a session tracking technique where the **Session ID is attached to every URL** so the server can identify the user.

It is used when **cookies are disabled** in the browser.

Real Life Example

Think of a **token system at a government office**:

You get a **token number** (Session ID)

You write that token number on **every form** you submit

Every counter knows it's YOU by that number

That token number written on every form = **URL Rewriting**.

Key Points

Works even if **cookies are disabled**.

Data is **visible in URL** — less secure.

Every link and form action must be **encoded**.

Uses `response.encodeURL()` method

URL looks like → `http://localhost:8080/shop;jsessionid=abc123xyz`

8. URL Rewriting

Important Methods :

Method	Meaning
<code>response.encodeURL(url)</code>	Adds session ID to URL
<code>response.encodeRedirectURL(url)</code>	Adds session ID to redirect URL

How URL Rewriting Works :



9. Other Servlet Capabilities

What are Other Servlet Capabilities?

Apart from basic request-response handling, servlets have many **extra/additional capabilities**.

List of Other Capabilities :

1. Request Dispatching

Servlet can **forward** request to another servlet or JSP.

2. Redirect

Servlet can **redirect** user to a completely different URL.

3. Servlet Filters

A **Filter** runs **before or after** a servlet. Used for — login check, logging, data compression.(Provide Security)

4. Servlet Listeners

Listeners watch for events like:

Session created or destroyed.

Application started or stopped.

10. Data Storage

What is Data Storage in Servlets?

Servlets can **store data** at different levels depending on how long and where the data needs to be available.

There are **3 main storage scopes**:

3 Types of Data Storage :



11. Servlets Concurrency

What is Servlets Concurrency?

When **multiple users** send requests to a servlet **at the same time** — this situation is called **Concurrency**.
The problem is — servlet creates only **ONE instance** for all users. So all users share the same servlet object.

Real Life Example

Think of a **single ATM machine**:

Many people want to use it at same time

But only one person can use it at a time

If two people use together → money calculation goes wrong!

Same problem happens in servlets with shared data.

12. Databases (MySQL) and Java Servlets :

What is Servlet-MySQL Connectivity?

Servlet can **connect to a MySQL database** to store, retrieve, update or delete data.

This is done using **JDBC (Java Database Connectivity)**.

Real Life Example :

Think of a **college admission form**:

Student fills form on website (Servlet)

Data gets saved in college database (MySQL)

Admin can view all students from database

Servlet is the **bridge** between user and database

Important JDBC Classes

Class	Meaning
DriverManager	Manages DB connections
Connection	Represents DB connection
Statement	Executes SQL queries
PreparedStatement	Executes safe SQL queries
ResultSet	Holds query results

13. XML Documents and Vocabularies

What is XML?

XML = eXtensible Markup Language

XML is a language used to **store and transport data** in a structured way that both humans and machines can read.

Simple Real Life Example

Think of XML like a **labeled tiffin box**:

Each compartment has a **label** (tag)

Label tells what's inside

Anyone can read and understand it easily.

Example :

```
<tiffin>  
  <roti>2</roti>  
  <sabji>aloo</sabji>  
  <dal>yes</dal>  
</tiffin>
```

13. XML Documents and Vocabularies

What are XML Documents?

An XML document is a **file that stores data using tags**.

Every XML document has:

Prolog → XML declaration at top

Root element → One main tag that wraps everything

Child elements → Tags inside root

Data → Actual content inside tags.

XML Document Example :

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <id>1</id>
    <name>Ram</name>
    <age>20</age>
    <branch>Computer</branch>
  </student>
  <student>
    <id>2</id>
    <name>Sita</name>
    <age>21</age>
    <branch>IT</branch>
  </student>
</students>
```

13. XML Documents and Vocabularies

What are XML Vocabularies?

XML Vocabulary means a **set of XML tags defined for a specific purpose**.

Different fields define their own XML tags — that collection is called a vocabulary.

Vocabulary	Used For
HTML	Web pages
SVG	Graphics
MathML	Mathematics
RSS	News feeds
WSDL	Web services
XHTML	Web pages (strict HTML)

Features of XML :

Self descriptive → tags describe the data

Platform independent → works on any OS

Human readable → easy to understand

Machine readable → easy to process

Extensible → you can create your own tags

14. XML Declaration & Structure

What is XML Declaration?

The **XML Declaration** is the **first line** of every XML document. It tells the browser — "This is an XML file!"

```
<?xml version="1.0" encoding="UTF-8"?>
```

XML Declaration — Parts Explained

Part	Meaning
<?xml	Start of declaration
version="1.0"	XML version being used
encoding="UTF-8"	Character set used
?>	End of declaration



What is XML Structure?

XML document has a **fixed structure** — like a tree.

Every XML document must follow this structure:

1. Prolog

```
<?xml version="1.0" encoding="UTF-8"?>
```

First line of XML.

Not mandatory but good practice.

Must be at very top if used.

2. Root Element

```
<students>
```

```
...
```

```
</students>
```

Every XML must have **exactly ONE root element**.

All other elements go inside root.

Like a tree trunk — everything branches from here.

3. Child Elements

```
<student>
```

```
    <name>Ram</name>
```

```
    <age>20</age>
```

```
</student>
```

Elements inside root are child elements.

Can have further nested children.

Must be properly closed.

4. Comments

```
<!-- This is a comment in XML -->
```

Same as HTML comments.

15. XML Namespaces

What is XML Namespace?

When you use XML from **two different sources**, there can be **tag name conflicts** — same tag name but different meaning.

Namespace solves this problem by giving each tag a **unique identity**.

The Problem — Without Namespace

```
<table>
```

```
  <tr><td>Student Data</td></tr>
```

```
</table>
```

```
<table>
```

```
  <length>4 feet</length>
```

```
  <width>2 feet</width>
```

```
</table>
```

Here <table> means:

First one → **HTML table** (rows & columns)

Second one → **Furniture table** (length & width)

15. XML Namespaces

Solution — With Namespace

```
<!-- HTML table with namespace -->
```

```
<html:table>
```

```
  <html:tr>
```

```
    <html:td>Student Data</html:td>
```

```
  </html:tr>
```

```
</html:table>
```

```
<!-- Furniture table with namespace -->
```

```
<furniture:table>
```

```
  <furniture:length>4 feet</furniture:length>
```

```
  <furniture:width>2 feet</furniture:width>
```

```
</furniture:table>
```

16. DOM Based XML Processing

What is DOM?

DOM = Document Object Model

DOM is a way to **read, access and modify** an XML document using Java code.

It loads the **entire XML file into memory** as a **tree structure** — then you can navigate and manipulate it.

Real Life Example

Think of XML like a **family tree**:

Grandfather is root

Children are child nodes

Grandchildren are further child nodes

DOM lets you **walk through this family tree** and find any member you want!

Key Points :

DOM loads **entire XML into memory**.

Creates a **tree of nodes**.

You can **read, add, update, delete** nodes.

Works well for **small XML files**.

Slow for large files — loads everything at once.

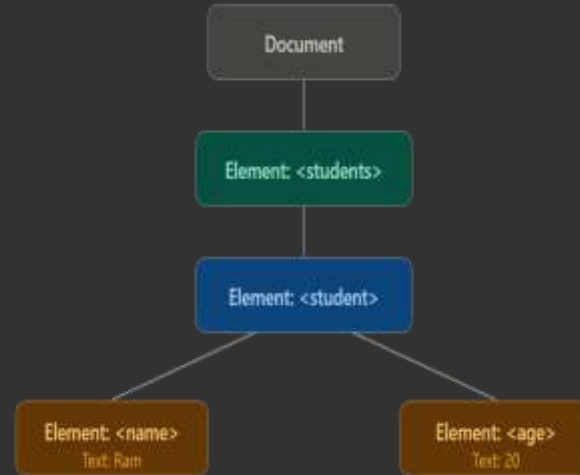
16. DOM Based XML Processing

DOM Tree — How XML looks in Memory

For this XML:

```
<students>  
  <student>  
    <name>Ram</name>  
    <age>20</age>  
  </student>  
</students>
```

DOM creates this tree:



17. Transforming XML Documents

What is XML Transformation?

XML Transformation means converting an XML document into **another format** like HTML, PDF or another XML using **XSLT**.

XSLT = eXtensible Stylesheet Language Transformations.

Key Points :

XML stores **data only** — no styling.

XSLT is used to **transform** XML into other formats.

XSLT is itself written in **XML format**.

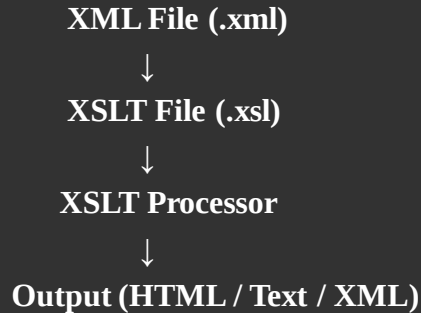
Output can be → **HTML, plain text, another XML**.

Uses **XPath** to navigate XML tree.

File extension → .xsl or .xslt

17. Transforming XML Documents

How Transformation Works :



Important XSLT Elements:

Element	Meaning
<xsl:stylesheet>	Root element of XSLT file
<xsl:template match="/">	Match root of XML
<xsl:for-each select="...">	Loop through elements
<xsl:value-of select="...">	Get value of element
<xsl:if test="...">	Condition check
<xsl:sort select="...">	Sort elements

18. DTD: Schema, Elements & Attributes

What is DTD?

DTD = Document Type Definition

DTD defines the **rules and structure** of an XML document. It tells — which elements are allowed, in what order and what attributes they can have.

Real Life Example:

Think of DTD like a **college admission form rules**:

Name field is **mandatory**

Age must be a **number**

Branch must be from a **fixed list**

If you break rules → form is **invalid**

DTD is those rules for XML!

Why DTD?

Validates XML document

Makes sure XML follows correct structure

Helps different systems understand XML data

Catches errors early

18. DTD: Schema, Elements & Attributes

Types of DTD

Type	Meaning
Internal DTD	Written inside XML file itself
External DTD	Written in separate .dtd file

1. Internal DTD Example :

```
<?xml version="1.0"?>
```

```
<!DOCTYPE students [  
  <!ELEMENT students (student+)>  
  <!ELEMENT student (name, age, branch)>  
  <!ELEMENT name (#PCDATA)>  
  <!ELEMENT age (#PCDATA)>  
  <!ELEMENT branch (#PCDATA)>  
  <!ATTLIST student id CDATA #REQUIRED>  

```

```
<students>  
  <student id="1">  
    <name>Ram</name>  
    <age>20</age>  
    <branch>Computer</branch>  
  </student>  
</students>
```

18. DTD: Schema, Elements & Attributes

2. External DTD Example

students.dtd file:

```
<!ELEMENT students (student+)>
<!ELEMENT student (name, age, branch)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
<!ELEMENT branch (#PCDATA)>
<!ATTLIST student id CDATA #REQUIRED>
```

XML file linking external DTD:

```
<?xml version="1.0"?>
<!DOCTYPE students SYSTEM "students.dtd">
```

```
<students>
  <student id="1">
    <name>Ram</name>
    <age>20</age>
    <branch>Computer</branch>
  </student>
</students>
```

18. DTD: Schema, Elements & Attributes

DTD Elements — Explained Simply

<!ELEMENT> — Defines an element

Declaration

`<!ELEMENT name (#PCDATA)>`

`<!ELEMENT student (name, age)>`

`<!ELEMENT students (student+)>`

`<!ELEMENT photo EMPTY>`

`<!ELEMENT note ANY>`

Meaning

name contains text

student has name then age

students has one or more student

photo has no content

note can have anything

Element Occurrence Symbols

Symbol	Meaning	Example
No symbol	Exactly once	(name)
+	One or more	(student+)
*	Zero or more	(student*)
?	Zero or one	(photo?)

18. DTD: Schema, Elements & Attributes

DTD Attributes :

<!ATTLIST> — Defines attributes

<!ATTLIST student id CDATA #REQUIRED>

Part	Meaning
ATTLIST	Attribute list keyword
student	Element name
id	Attribute name
CDATA	Attribute type (character data)
#REQUIRED	Attribute is mandatory

Attribute Types

Type	Meaning
CDATA	Plain text
ID	Unique identifier
IDREF	Reference to an ID
NMTOKEN	Valid XML name

18. DTD: Schema, Elements & Attributes

DTD vs XML Schema

Point	DTD	XML Schema
File extension	.dtd	.xsd
Written in	Special DTD syntax	XML format
Data types	Limited	Rich data types
Namespace support	No	Yes
Easy to learn	Yes	Harder
More powerful	No	Yes

19. Introduction to AJAX

What is AJAX?

AJAX = Asynchronous JavaScript And XML

AJAX is a technique that allows a webpage to **send and receive data from server without reloading the entire page.**

Real Life Example

Think of **Google Search suggestions:**

You type "Web T..." in search box

Suggestions appear **instantly** below

Page does **NOT reload**

Data comes from server **in background**

That is AJAX!

What does AJAX stand for?

Letter	Meaning
A	Asynchronous — works in background
J	JavaScript — used to send request
A	and
X	XML — data format (now JSON also used)

19. Introduction to AJAX

Without AJAX vs With AJAX

Without AJAX

Fill form → Submit

Entire page reloads

Slow & annoying

User waits for reload

With AJAX

Fill form → Submit

Only part of page updates

Fast & smooth

User sees instant update

Key Points :

AJAX is **not a programming language**.

Uses **JavaScript + XMLHttpRequest object**.

Data exchange in **XML or JSON format**.

Page updates **without full reload**.

Makes web apps **faster and smoother**.

Examples → Gmail, Google Maps, Facebook feed.

19. Introduction to AJAX

Advantages of AJAX

Page does not reload — **faster**

Better **user experience**

Less **bandwidth** usage

Works in **background**

Only **required part** of page updates.

Disadvantages of AJAX

Browser back button may not work properly

Search engines cannot index AJAX content easily

Requires **JavaScript enabled** in browser

Security issues if not handled properly.

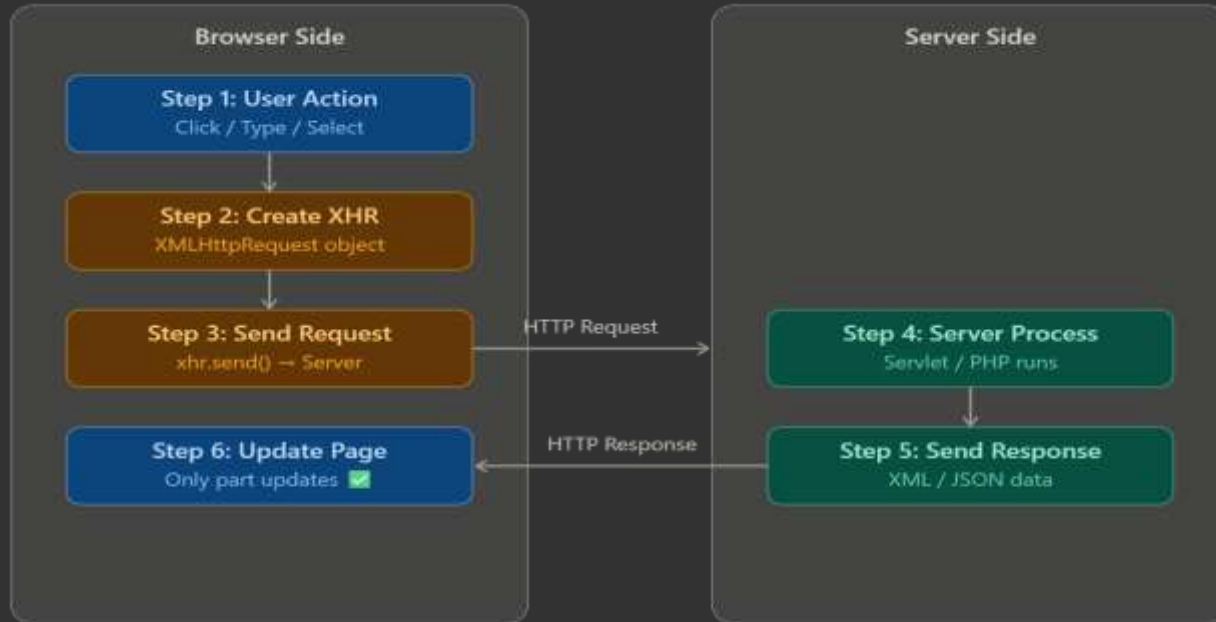
20. Working of AJAX

How AJAX Works?

AJAX works using a special JavaScript object called **XMLHttpRequest (XHR)**.

This object sends request to server **in background** and gets response **without reloading page**.

AJAX Working :





Thank You!



Like, Share & Subscribe

<https://youtube.com/@thevijayacademy?si=uYwlDuAxyZfLsiei>

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy